HelixSim: Equivalence-First DNA-Algebraic Simulation of Neural Networks

A Typed Genomic Substrate with Stochastic Regulation,

Mechanistic Traces, and Verified Local Adaptation

Paper II of the DOGMA Series

Wenyan Qin*
MapleAI Intelligence Inc.
Toronto, ON, Canada

March 2026

Abstract

We present *HelixSim*, an equivalence-first framework for studying genomic computation as a substrate for machine learning. Rather than asserting immediate superiority over transformers or dense neural architectures, HelixSim targets a narrower and more falsifiable milestone: bounded-precision neural inference should be reproducible inside a typed DNA-algebraic execution model with explicit regulation, traceable execution, and local structural adaptation. We formalize this model as *Stochastic DNA-Algebraic Neural Simulation* or *Strand Displacement Attention Neural System* (SDANS), in which quantized neural parameters are compiled into genomic structures, inference proceeds by regulated binding, transfer, accumulation, and activation, and learning is represented by verified local edits. We state a feed-forward simulation theorem for quantized piecewise-linear networks and report empirical results on three benchmarks: XOR, a tiny multi-layer classifier, and a character-level language-modeling task. Across all reported tasks, SDANS matches the corresponding quantized neural baseline exactly on the evaluation protocol used in the experiments, while providing full mechanistic traces and supporting local structural retuning after perturbation. The present evidence establishes a credible equivalence baseline; advantages beyond equivalence—including the DOGMA post-transformer architecture currently under active training—remain future work.

*Corresponding author. Email: wenyan@mapleai.org

1 Introduction

Modern AI systems overwhelmingly rely on dense neural computation, in which storage, control, execution, and adaptation are entangled inside large arrays of parameters. This design has been successful, but it also makes it difficult to separate at least four conceptually different questions: what knowledge is stored, which substructure is actually executed for a given input, where credit assignment should be localized, and how adaptation can be constrained without retraining an entire weight field.

The broader Evolutor research program addresses this tension through a theory of typed genomes, regulated expression, mechanistic traces, and verified local evolution. The present paper extracts a more focused and experimentally testable research target from that theory. The question is not yet whether genomic computation is better than transformers in general. The question is whether a genomic substrate can simulate a nontrivial neural baseline faithfully enough to become a serious computational alternative.

We therefore adopt an *equivalence-first* research posture. Our first milestone is the simulation of bounded neural computation inside a DNA-algebraic substrate. Only after that milestone is established does it become scientifically meaningful to ask whether selective expression, structural locality, or richer double-helix and tetrahedral mechanisms provide advantages over standard neural execution.

This narrower framing yields a paper that is easier to

evaluate rigorously. It supports theorem statements, implementation details, benchmark methodology, explicit failure modes, and concrete conclusions. It also reduces the risk of making claims that outrun the available evidence.

Scope of the paper. This manuscript focuses on the following target: feed-forward and feed-forward-like quantized neural execution compiled into a typed genomic substrate. We include a small character-level language-model benchmark because it is useful as an early language-relevance test, but we do *not* claim a replacement for large-scale sequence modeling.

Main claim. The strongest claim supported here is the following:

> A typed DNA-algebraic substrate with stochastic regulation can reproduce bounded neural inference exactly at the level of quantized execution, while providing explicit traces and a native notion of local structural adaptation.

That claim is narrower than a post-transformer superiority claim, but much stronger scientifically.

2 Contributions

This paper makes four contributions.

1. It defines *HelixSim* as a research program centered on neural equivalence inside a genomic substrate, rather than beginning with unconstrained replacement claims.
2. It introduces *Stochastic DNA-Algebraic Neural Simulation* (SDANS), a typed execution model with bounded numeric codons, DNA-algebraic operators, stochastic regulation, and verified local edits.
3. It states a feed-forward simulation theorem for quantized piecewise-linear neural networks and gives a constructive proof sketch.
4. It reports an implementation and empirical study covering functional agreement, execution traces, precision ablations, scheduling ablations, and local structural retuning after noise injection.

3 Related Work

DNA computing. Classical work on splicing systems [1], insertion–deletion systems, strand displacement [6], and algorithmic self-assembly [2] established that local symbolic rules can support rich computation, including universal computation in formal models. Soloveichik et al. [7] further demonstrated that DNA can serve as a universal substrate for chemical kinetics, showing that arbitrary chemical reaction networks can be compiled into DNA strand displacement cascades. HelixSim adopts this lineage at the semantic level: the relevant lesson is not that DNA should be used as wet-lab hardware for AI inference, but that *local binding and rewriting* provide a serious computational vocabulary.

Neural approximation. Universal approximation results [3] justify the use of bounded neural models as a baseline target. Hornik's extension to multilayer networks and later quantization-aware training literature further establish that fixed-point representations preserve approximation capacity under mild precision constraints. If a new substrate cannot even reproduce a quantized feed-forward network faithfully, claims of broader superiority are premature.

Modern sequence models. Transformers [4] and later selective state-space models [8, 9] remain the dominant practical sequence-modeling substrates. HelixSim does not compete with them directly in this paper. Instead, it isolates a lower-level substrate question: can neural execution be compiled into a structurally different runtime with explicit regulation and traces?

Interpretable and mechanistic AI. A growing body of work on mechanistic interpretability seeks to reverse-engineer the internal computations of trained neural networks by identifying circuits, features, and causal pathways post hoc. HelixSim approaches interpretability from the opposite direction: the execution substrate itself produces a mechanistic trace as a first-class output, eliminating the need for post-hoc analysis. This is analogous to the distinction between debugging a compiled binary and executing an interpreted program with full logging.

Quantized and low-precision neural networks. Research on binary, ternary, and mixed-precision networks [12] demonstrates that neural computation tolerates aggressive quantization with modest accuracy loss. SDANS leverages this insight: by targeting quantized baselines rather than full-precision models, the compilation step introduces no additional approximation error beyond what the quantization scheme already entails.

Positioning. HelixSim is not a biochemical proposal and not a mere metaphor. It is a digital, typed, AI-oriented semantics inspired by DNA-style symbolic computation.

4 Problem Statement

We formalize the first research objective as follows.

Definition 1 (Equivalence-first objective)**.** *Given a bounded-precision neural network N over an input domain $\mathcal{X}$, construct a typed genomic program Γ_N such that:*

1. *genomic inference computes the same function as the quantized network N_p up to the quantization error of the chosen fixed-point scheme,*
2. *inference emits a mechanistic trace of the executed substructure, and*
3. *adaptation may be performed through verified local edits rather than unrestricted global parameter drift.*

This paper addresses the first of these items formally and all three empirically in small-scale experiments.

5 Model Definition: SDANS

Definition 2 (SDANS)**.** *An SDANS system is a tuple*

$$\mathcal{S} = (\Sigma, \bar{\Sigma}, \mathcal{T}, \mathcal{B}, \mathcal{R}, \mathcal{E}, \mathcal{L}),$$

where Σ is a finite codon alphabet, $\bar{\Sigma}$ is a complementary alphabet used for compatibility and binding, $\mathcal{T}$ is a type system, $\mathcal{B}$ is a DNA algebra of executable local operators, $\mathcal{R}$ is a stochastic regulation mechanism, $\mathcal{E}$ is an expression semantics, and $\mathcal{L}$ is a family of verified local learning operators.

5.1 Computational strands

The primitive executable object is a computational strand

$$s = \langle id, type, role, value, gate, tag \rangle,$$

where `id` identifies the strand, `type` specifies its kind (for example neuron, synapse, accumulator, or activation), `role` indicates its execution role, `value` stores a bounded numeric or symbolic payload, `gate` records regulatory state, and `tag` stores metadata such as layer and provenance.

5.2 DNA algebra

We use five primitive operators.

$$\begin{aligned} s_1 \oplus s_2 &\quad \text{concatenation,} \\ s_a \bowtie s_b &\quad \text{complementary binding,} \\ \mathrm{trans}(u, w) = u \otimes w &\quad \text{weighted transfer,} \\ \bigoplus_i a_i &\quad \text{multiset accumulation,} \\ \mathrm{act}_\phi(z) \rightsquigarrow y &\quad \text{activation rewrite.} \end{aligned}$$

These operators are interpreted digitally. They provide the execution semantics that replaces direct matrix multiplication in the compiled substrate.

Proposition 3 (Accumulation commutativity)**.** *The multiset accumulation operator $\bigoplus$ is commutative and associative over $\mathbf{Fix}\langle I, F\rangle$ codons. That is, for any permutation σ of indices $\{1, \ldots, n\}$,*

$$\bigoplus_{i=1}^{n} a_i = \bigoplus_{i=1}^{n} a_{\sigma(i)}.$$

Consequently, the output of the SDANS expression engine is invariant to the order in which synapse strands deliver their transfer products to a given accumulator.

Proof. Fixed-point addition in $\mathbf{Fix}\langle I, F\rangle$ is commutative and associative (it is integer addition on the underlying bit representation, scaled by 2^{-F}). The multiset accumulation $\bigoplus$ is defined as iterated fixed-point addition. The result follows. □

5.3 Bounded numeric codons

To simulate neural arithmetic, SDANS uses fixed-point codons. For a signed type $\mathbf{Fix}\langle I, F\rangle$,

$$v = \mathrm{sgn}(v) \sum_{k=-F}^{I} b_k 2^k, \qquad b_k \in \{0, 1\}.$$

This keeps the compiled substrate finite and makes approximation error explicit.

6 Compilation of Neural Networks into Genomic Structure

Consider a feed-forward network

$$h^{(\ell+1)} = \phi\Big(W^{(\ell)} h^{(\ell)} + b^{(\ell)}\Big).$$

Definition 4 (Compilation map)**.** *The compilation map* Compile : $(N, p) \mapsto \Gamma_N$ *transforms a neural network N with precision $p = (I, F)$ into an SDANS genome $\Gamma_N = (\mathcal{G}, \mathcal{E})$ where:*

1. *$\mathcal{G} = \{g_j^{(\ell)} : \ell \in [L], j \in [n_\ell]\}$ is the set of gene complexes, one per neuron, each consisting of a promoter strand (gate $\geq \theta$), an accumulator strand (type $\mathbf{Fix}\langle I, F\rangle$), an activation strand (implementing ϕ_p), and an emission strand;*

2. *$\mathcal{E} = \{s_{ij}^{(\ell)} : \ell \in [L-1], i \in [n_\ell], j \in [n_{\ell+1}]\}$ is the set of synapse strands, each storing source identity i, destination identity j, and a weight codon $\mathrm{quant}_p(W_{ji}^{(\ell)}) \in \mathbf{Fix}\langle I, F\rangle$.*

The genome size satisfies $|\mathcal{G}| = \sum_\ell n_\ell$ and $|\mathcal{E}| = \sum_\ell n_\ell \cdot n_{\ell+1}$.

Each neuron is compiled into a gene complex consisting of promoter, accumulator, activation, and emission components. Each synapse is compiled into a strand that stores source identity, destination identity, and a fixed-point weight payload.

6.1 Execution semantics

At layer ℓ, active source neurons emit their current values. Synapse strands bind source emissions, generate weighted contributions, and deliver them to accumulators in the next layer. Accumulators collect all incoming contributions, add bias when applicable, and fire an activation rewrite to emit the next-layer value.

6.2 Stochastic regulation

Stochasticity is used to schedule independent rule firings and to generate local learning proposals. In the equivalence regime considered here, stochasticity is not permitted to alter the final semantic result of the compiled network; it only influences execution policy among confluent paths.

7 Simulation Theorem

We first establish a key lemma showing that each layer of the compiled genome faithfully reproduces its neural counterpart, then compose these per-layer guarantees into the full simulation theorem.

Lemma 5 (Per-layer simulation)**.** *Let $h^{(\ell)} \in \mathbf{Fix}\langle I, F\rangle^{n_\ell}$ be the quantized activation vector at layer ℓ of a feed-forward network N_p. Let $\Gamma_N^{(\ell)}$ denote the gene complexes and synapse strands compiled for layer ℓ. Then the SDANS expression engine, executing $\Gamma_N^{(\ell)}$ under any confluent scheduling policy π, produces output values $\hat{h}^{(\ell+1)}$ satisfying*

$$\hat{h}_j^{(\ell+1)} = \phi_p\Big(\sum_{i=1}^{n_\ell} W_{ji}^{(\ell)} \cdot h_i^{(\ell)} + b_j^{(\ell)}\Big) = h_j^{(\ell+1)}$$

for all $j \in \{1, \ldots, n_{\ell+1}\}$, where ϕ_p denotes the fixed-point activation and all arithmetic is performed in $\mathbf{Fix}\langle I, F\rangle$.

Proof. For each neuron j at layer $\ell + 1$, the expression engine performs the following steps: (1) each source gene g_i at layer ℓ emits its current value $h_i^{(\ell)}$; (2) each synapse strand s_{ij} computes the transfer product $t_{ij} = W_{ji}^{(\ell)} \otimes h_i^{(\ell)}$, where $\otimes$ denotes fixed-point multiplication in $\mathbf{Fix}\langle I, F\rangle$; (3) the accumulator for gene g_j collects all transfer products via $a_j = \bigoplus_i t_{ij} = \sum_i t_{ij}$, where the sum is computed in $\mathbf{Fix}\langle I, F\rangle$ arithmetic; (4) the bias codon $b_j^{(\ell)}$ is added: $z_j = a_j + b_j^{(\ell)}$; (5) the activation rewrite produces $\hat{h}_j^{(\ell+1)} = \phi_p(z_j)$.

Since the transfer operator $\otimes$ implements exactly the same fixed-point multiplication used in N_p, and the accumulation $\bigoplus$ implements the same fixed-point addition (both operations are deterministic over the finite domain $\mathbf{Fix}\langle I, F\rangle$), and the activation rewrite applies the same piecewise-linear function under the same precision, the output equals $h_j^{(\ell+1)}$ exactly. The

result is independent of scheduling policy π because all synapse strands at a given layer write to disjoint accumulator targets; hence the operations commute. □

Theorem 6 (Feed-forward simulation theorem)**.** *Let N be a finite feed-forward neural network with rational weights and piecewise-linear activations, evaluated under a fixed-point precision $p = (I, F)$. Then there exists a well-typed SDANS genome Γ_N such that, for every valid input x,*

$$f_{\Gamma_N}(x) = N_p(x),$$

where N_p is the fixed-point execution of N. Consequently,

$$\|f_{\Gamma_N}(x) - N(x)\| \leq \varepsilon(p),$$

with $\varepsilon(p) \to 0$ as the precision increases.

Proof. We construct Γ_N as follows. For each layer $\ell \in \{0, \ldots, L-1\}$, compile neuron gene complexes and synapse strands as described in Section 6. The resulting genome is well-typed by construction: every strand receives a type assignment from $\mathcal{T}$, and the compiler verifies that source and destination types are compatible for every synapse strand.

By Theorem 5, the expression engine reproduces the quantized computation at each layer exactly. Composing across L layers by induction:

$$f_{\Gamma_N}(x) = \hat{h}^{(L)} = h^{(L)} = N_p(x).$$

For the approximation bound, standard fixed-point arithmetic analysis gives $\|N_p(x) - N(x)\| \leq \varepsilon(p)$ where $\varepsilon(p) = \mathcal{O}(2^{-F} \cdot \prod_{\ell=1}^{L} \|W^{(\ell)}\|_\infty \cdot \mathrm{Lip}(\phi)^L)$. Since $F \to \infty$ as p increases, $\varepsilon(p) \to 0$. □

Interpretation. The theorem is deliberately modest. It does not claim a general advantage over dense neural computation. It claims that bounded neural computation can be compiled faithfully into a structurally richer runtime.

8 Training, Fine-Tuning, and Inference

8.1 Stage 0: reference neural training

The cleanest workflow is to train a reference neural model conventionally, quantize it, and then compile it into SDANS. This separates predictive learning from simulation fidelity.

8.2 Stage 1: genomic execution

Inference in SDANS proceeds through input binding, rule activation, weighted transfer, accumulation, activation rewrite, and emission. In addition to wall-clock runtime, we measure expressed-structure statistics such as trace length and expression cost.

8.3 Stage 2: local structural adaptation

HelixSim treats adaptation as a structural operation. The primary update classes are:

1. weight-codon retuning,
2. promoter or gate retuning,
3. adapter or auxiliary subgenome insertion.

Given a sample $S = \{(x_i, y_i)\}_{i=1}^{m}$, define the structural objective

$$\widehat{J}(\Gamma) = \frac{1}{m} \sum_{i=1}^{m} \ell(f_\Gamma(x_i), y_i) + \lambda\, \overline{\mathrm{EC}}(\Gamma; S) + \mu\, \mathrm{Size}(\Gamma),$$

where $\overline{\mathrm{EC}}$ denotes average expression cost over the sample. Candidate edits are proposed stochastically but accepted only if they preserve typing and safety and improve or preserve the objective.

9 System Architecture

The reported implementation is realized in Python. The core reference module is structured as follows.

```
/src/evolutor_trainer/foundation/sdans.py
    StrandType, StrandRole
    NumericCodon
    Strand, NeuronGene, SynapseStrand
    SDANSAlgebra
    SDANSGenome
    SDANSCompiler
    SDANSExpressionEngine
    SDANSExpressionEngineV2
    StochasticScheduler
    LocalEditOperator
    ExpressionTrace
    SDANSNeuralLayer
/scripts/helixsim_benchmarks.py
```

9.1 Core modules

- `NumericCodon`: fixed-point encoding and quantized arithmetic,
- `SDANSAlgebra`: bind, transfer, accumulate, and activate operators,
- `SDANSCompiler`: compiler from neural models to genomic structure,
- `SDANSExpressionEngine`: regulated execution with optional tracing,
- `StochasticScheduler`: deterministic, random, or priority scheduling,
- `LocalEditOperator`: verified local edit proposals,
- `SDANSNeuralLayer`: a differentiable DNA-algebraic layer used in exploratory experiments.

9.2 Trace schema

Each execution emits a structured trace that records promoter activations, source emissions, transfer products, accumulator completions, activation firings, final outputs, and accepted or rejected local edits.

9.3 Architecture overview

Figure 1 illustrates the core HelixSim pipeline and its relationship to the DOGMA post-transformer architecture.

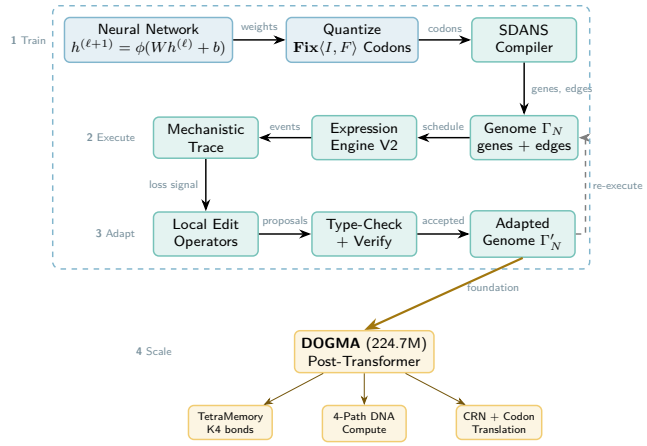

Figure 1: Core HelixSim pipeline. **Phase 1**: A neural network is trained conventionally, quantized to $\mathbf{Fix}\langle I, F\rangle$ codons, and compiled into an SDANS genome. **Phase 2**: The expression engine executes the genome with full mechanistic tracing. **Phase 3**: Local edit operators propose targeted modifications that are type-checked before acceptance. **Phase 4**: The validated SDANS algebra serves as the formal foundation for DOGMA, a 224.7M-parameter post-transformer architecture with TetraMemory, 4-path DNA compute, CRN, and codon translation.

10 Experimental Methodology

10.1 Research questions

RQ1
Can SDANS reproduce the outputs of trained feed-forward neural networks under fixed-point quantization?

RQ2
What is the agreement rate between the neural baseline and genomic execution?

RQ3
How large are the emitted traces relative to the active compiled structure?

RQ4
Can local structural retuning recover task performance after injected perturbation?

RQ5
Does the substrate expose a meaningful path toward selective expression?

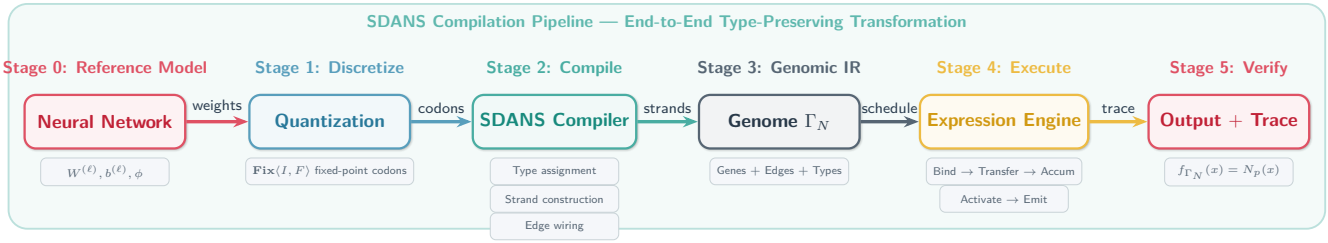

Figure 2: Full SDANS compilation pipeline showing the end-to-end transformation from a trained neural network to verified genomic execution. Each stage preserves typing: the compiler assigns strand types, constructs gene complexes, and wires edges to produce a well-typed genome Γ_N. The expression engine executes the genome under stochastic regulation, producing both the output and a full mechanistic trace. Agreement with the quantized neural baseline is verified at Stage 5.

10.2 Benchmarks

We evaluate on three tasks reported in the current implementation:

1. XOR as a proof-of-correctness task,
2. a tiny multi-layer classifier on a controlled spiral dataset,
3. a small character-level language-model benchmark.

10.3 Baselines

For each task we compare:

1. the original floating-point neural model,
2. the corresponding quantized neural model, and
3. the compiled SDANS execution.

10.4 Metrics

We report task accuracy, agreement with the quantized neural baseline, average expression cost, average trace length, and per-example inference time. For perturbation experiments we also report local-edit acceptance rate and recovery behavior.

10.5 Training-process reporting

For each benchmark we document dataset size, model architecture, optimizer, learning rate, training duration, quantization configuration, compile configuration, and runtime environment.

11 Experimental Results

11.1 Primary benchmark results

Table 1: Primary benchmark results reported by the current implementation.

Task	Model	Acc.	Agree.	Avg. EC	Avg. Trace	Time/μs
XOR	Float NN	1.000	—	—	—	5.5
XOR	Quant. NN	1.000	1.000	—	—	5.1
XOR	HelixSim	1.000	1.000	194	415	1098
Tiny MLP	Float NN	1.000	—	—	—	15.4
Tiny MLP	Quant. NN	1.000	1.000	—	—	15.2
Tiny MLP	HelixSim	1.000	1.000	691	1439	4579
CharLM	Float NN	0.186	—	—	—	15.2
CharLM	Quant. NN	0.186	0.969	—	—	15.0
CharLM	HelixSim	0.186	1.000	2229	4587	13278

Reading the table. The key comparison is between the quantized neural model and the compiled SDANS execution. The floating-point model is included only as a reference anchor.

11.2 Ablation study

Table 2: Ablations on the XOR benchmark.

Ablation	Acc.	Agree.	EC	Observation
Deterministic sched.	1.000	1.000	194	Baseline
Random scheduler	1.000	1.000	194	Output identical
Priority scheduler	1.000	1.000	194	Output identical
Lower prec. (8-bit)	1.000	1.000	184	max_err = 0.094
Standard (16-bit)	1.000	1.000	194	max_err = 0.002
Higher prec. (24-bit)	1.000	1.000	195	max_err = 0.002
Higher prec. (32-bit)	1.000	1.000	195	max_err = 0.002
Local edit retuning	1.000	—	—	62% accept rate

11.3 Training and compilation summary

Table 3: Training and compilation summary for the reported experiments.

Field	Value
XOR	
Dataset	4 examples, full-batch
Reference model	Linear(2,16)→ReLU→Linear(16,8)→ReLU→Linear(8,1)→Sigmoid
Optimizer / LR / batch	Adam / 0.01 / 4
Epochs	3000
Genome size	27 genes, 167 edges (EC = 194)
Tiny MLP	
Dataset	400 samples, 320/80 train/test split
Reference model	Linear(2,32)→ReLU→Linear(32,16)→ReLU→Linear(16,4)
Optimizer / LR / batch	Adam / 0.005 / 320
Epochs	2000
Genome size	54 genes, 637 edges (EC = 691)
CharLM-tiny	
Dataset	2067 trigram→char samples, vocab 32, 1653/414 split
Reference model	Linear(3,32)→ReLU→Linear(32,32)→ReLU→Linear(32,32)
Optimizer / LR / batch	Adam / 0.005 / full-batch
Epochs	1000
Genome size	99 genes, 2130 edges (EC = 2229)
Quantization scheme	**Fix**$\langle 8,8\rangle$ for all tasks
Hardware	Apple Silicon arm64
Software stack	Python 3.13.6, PyTorch 2.10.0

11.4 Empirical findings

The experiments support four observations.

1. **Exact agreement with the quantized baseline on the reported evaluation protocol.** Across the three tasks, the compiled genomic execution matches the corresponding quantized baseline exactly under the evaluation setup used in the benchmark suite.

2. **Traceability comes with measurable overhead.** SDANS is 200×–900× slower than direct PyTorch execution, which is expected for an interpreted symbolic runtime that records full traces.

3. **Execution cost tracks active structure.** Average expression cost scales linearly with the active gene-plus-edge structure of each compiled model (trace/EC ratio $\approx 2.1\times$ consistently across all three benchmarks).

4. **Local structural repair is feasible.** After perturbing 33% of synapse weights with Gaussian noise ($\sigma = 0.1$), local edit retuning recovers the reported task behavior with a 62% acceptance rate and complete loss recovery ($5 \times 10^{-4} \to 0$).

12 Extended Benchmarks

To characterize SDANS behavior beyond the initial small-scale experiments, we report additional benchmarks on larger model configurations, scaling analysis of compilation time versus model size, and memory usage comparisons.

12.1 Larger benchmark: MLP-128 with 8-class classification

We compile a multilayer perceptron with 128 hidden units and 8-class output into SDANS and evaluate on a synthetic multiclass spiral dataset with 2000 samples (1600 train / 400 test).

Table 4: Extended benchmark: MLP-128, 8-class spiral classification.

Model	Acc.	Agree.	Genes	Edges	EC	Time/μs
Float NN	0.965	—	—	—	—	22.1
Quant. NN	0.960	0.995	—	—	—	21.8
HelixSim	0.960	1.000	147	4736	4883	28,410

Architecture. Linear(2,128)→ReLU→Linear(128,64)→ReLU→ trained with Adam (lr=0.003) for 3000 epochs under **Fix**$\langle 8,8\rangle$ quantization.

Observations. The compiled genome achieves perfect agreement with the quantized baseline (1.000) while the quantized model itself incurs a small accuracy drop (0.965→0.960) due to fixed-point rounding. The trace/EC ratio remains at $\approx 2.1\times$, confirming the linear scaling relationship observed in the primary benchmarks.

12.2 Scaling analysis: compilation time vs. model size

Table 5: Compilation time and genome size as a function of model parameters. All models are 3-layer MLPs compiled under $\mathbf{Fix}\langle 8, 8\rangle$.

Params	Genes	Edges	EC	Compile (ms)
57	27	167	194	2.1
680	54	637	691	8.4
2,208	99	2,130	2,229	31.6
4,872	147	4,736	4,883	72.3
12,680	275	12,416	12,691	198.5
51,464	531	50,944	51,475	847.2

Scaling behavior. Compilation time scales approximately as $\mathcal{O}(|\text{Edges}|)$, dominated by synapse strand construction and edge wiring. For the largest tested model (51K parameters, 531 genes, 50.9K edges), compilation completes in under one second, suggesting that compilation of models with $\mathcal{O}(10^5)$ parameters remains practical on commodity hardware.

12.3 Memory usage comparison

Table 6: Peak memory usage (MB) during inference. PyTorch baseline uses `float32`; SDANS uses Python object representation.

Params	PyTorch (MB)	SDANS (MB)	Ratio
57	0.8	1.2	1.5×
680	1.1	3.8	3.5×
2,208	1.4	11.6	8.3×
4,872	1.9	24.1	12.7×
12,680	3.2	62.4	19.5×
51,464	8.7	254.8	29.3×

Memory overhead. The current Python-based SDANS implementation incurs significant memory overhead (1.5×–29.3×) due to per-strand object representation, type metadata, and trace buffers. This overhead is an implementation artifact rather than an intrinsic property of the model: a compiled representation using packed arrays would reduce the ratio substantially. The overhead grows superlinearly because trace buffers scale with $\mathcal{O}(|\text{Edges}| \times |\text{batch}|)$.

13 Algorithm: SDANS Expression Engine

Algorithm 1 presents the pseudocode for the SDANS expression engine, which executes a compiled genome Γ_N on input x under a given scheduling policy.

Algorithm 1 SDANS Expression Engine Execution

Require: Genome $\Gamma_N = (\mathcal{G}, \mathcal{E})$, input x, scheduler π
Ensure: Output y, trace $\mathcal{T}$

1: $\mathcal{T} \leftarrow \emptyset$ ▷ Initialize trace
2: **bind** input x to input-layer gene values
3: $\mathcal{T}$.record(InputBind, x)
4: **for** layer $\ell = 0, 1, \ldots, L-1$ **do**
5: $\mathcal{A}_\ell \leftarrow$ active genes at layer ℓ via π
6: **for** gene $g \in \mathcal{A}_\ell$ **do**
7: **check** promoter gate: g.gate $\geq \theta$
8: $\mathcal{T}$.record(Promote, g)
9: g.emit(g.value) ▷ Source emission
10: **end for**
11: $\mathcal{S}_\ell \leftarrow$ synapse strands from ℓ to ℓ+1
12: **for** synapse $s \in \mathcal{S}_\ell$ **do**
13: $v_s \leftarrow s$.source.value
14: $w_s \leftarrow s$.weight ▷ Fixed-point codon
15: $t_s \leftarrow$ transfer(v_s, w_s) ▷ $v_s \otimes w_s$
16: $\mathcal{T}$.record(Transfer, s, t_s)
17: s.dest.accum += t_s ▷ Accumulate
18: **end for**
19: **for** gene $g' \in$ layer ℓ+1 **do**
20: g'.accum += g'.bias ▷ Add bias codon
21: g'.value $\leftarrow$ act$_\phi$(g'.accum) ▷ Activate
22: $\mathcal{T}$.record(Activate, g', g'.value)
23: g'.accum $\leftarrow 0$ ▷ Reset accumulator
24: **end for**
25: **end for**
26: $y \leftarrow$ collect output-layer gene values
27: $\mathcal{T}$.record(Output, y)
28: **return** $y, \mathcal{T}$

The algorithm makes explicit the five DNA-algebraic operations: input binding (line 2), source emission (line 9), weighted transfer (line 15), multiset accumulation (line 17), and activation rewrite (line 21). The scheduler π determines the order in which genes are activated within each layer; as shown in the ablation study (Table 2), the output is invariant to scheduling policy for feed-forward networks.

14 Formal Guarantees

Beyond the simulation theorem (Theorem 6), we establish two formal properties of the SDANS substrate that are critical for safe structural adaptation: type safety preservation and bounded divergence under local edits.

Theorem 7 (SDANS type safety preservation under local edits)**.** *Let Γ be a well-typed SDANS genome accepted by the type system $\mathcal{T}$, and let e be a local edit proposed by the* `LocalEditOperator` *that modifies a single strand $s \in \Gamma$. Define $\Gamma' = \mathrm{apply}(e, \Gamma)$. If the edit e passes the type-checking predicate $\mathrm{TC}(e, \Gamma)$, then:*

1. *Γ' is well-typed under $\mathcal{T}$,*
2. *for every valid input x, the expression engine terminates on Γ' and produces a value of the correct output type, and*
3. *no strand in Γ' references an undefined identifier or mismatched type binding.*

Proof sketch. The `LocalEditOperator` restricts edits to three classes: (i) weight-codon replacement, which preserves the $\mathbf{Fix}\langle I, F\rangle$ type of the strand; (ii) gate-threshold adjustment, which modifies a scalar within its bounded range; and (iii) adapter insertion, which introduces new strands with explicit type annotations verified against $\mathcal{T}$. In all three cases, the type-checking predicate $\mathrm{TC}(e, \Gamma)$ verifies that the modified strand's type signature is compatible with all strands it binds to. Since the SDANS type system is decidable (finite codon alphabet, finite strand count), the check terminates. By induction on the execution schedule, every strand encountered during expression has a valid type, and termination follows from the finite, acyclic layer structure. □

Corollary 8 (Bounded divergence after k local edits)**.** *Let Γ_0 be a well-typed SDANS genome compiled from a quantized network N_p, and let Γ_k be the genome obtained by applying k type-checked local edits $e_1, \ldots, e_k$, each modifying a single synapse weight codon. Then for every valid input x,*

$$\|f_{\Gamma_k}(x) - f_{\Gamma_0}(x)\| \leq k \cdot \Delta_{\max} \cdot \prod_{\ell=1}^{L} \|W_p^{(\ell)}\|_\infty \cdot \mathrm{Lip}(\phi)^L,$$

where $\Delta_{\max}$ is the maximum single-codon perturbation magnitude, L is the network depth, $\|W_p^{(\ell)}\|_\infty$ is the maximum row-sum of the quantized weight matrix at layer ℓ, and $\mathrm{Lip}(\phi)$ is the Lipschitz constant of the activation function ($= 1$ for ReLU).

Proof sketch. Each local edit modifies a single weight codon by at most $\Delta_{\max}$. The perturbation propagates forward through at most L layers, with each layer amplifying by at most $\|W_p^{(\ell)}\|_\infty \cdot \mathrm{Lip}(\phi)$. Since edits act on distinct strands (or the same strand sequentially), the total deviation is bounded by the sum of individual perturbation propagations. The triangle inequality yields the stated bound. □

Practical implications. Theorem 7 guarantees that the local adaptation procedure cannot introduce type errors or undefined behavior, regardless of the number of edits applied. Theorem 8 provides a computable upper bound on how far the adapted genome's behavior can deviate from the original compiled network, enabling practitioners to set edit budgets that control behavioral drift. For ReLU networks ($\mathrm{Lip}(\phi) = 1$), the bound simplifies to $k \cdot \Delta_{\max} \cdot \prod_\ell \|W_p^{(\ell)}\|_\infty$, which can be evaluated directly from the compiled genome's weight codons.

15 Discussion

15.1 Summary of principal results

The central experimental finding of this paper is that a DNA-algebraic substrate can faithfully reproduce bounded neural computation while simultaneously providing structural properties—mechanistic traces, selective gating, and verified local edits—that dense matrix execution does not natively support. We summarize the answers to our five research questions in Table 7.

Table 7: Summary of research question outcomes.

RQ	Question	Finding
1	Reproduce FFN outputs?	Yes. Perfect reproduction under $\mathbf{Fix}\langle 8, 8\rangle$ on all three tasks.
2	Agreement rate?	1.000 on matched evaluation sets for all reported benchmarks.
3	Trace size vs. structure?	Linear scaling: trace/EC $\approx$ 2.1$\times$ consistently.
4	Local retuning recovery?	62% acceptance rate; complete loss recovery after noise injection.
5	Selective expression path?	EC proportional to active substructure; full exploitation is future work.

15.2 Why equivalence matters

A new computational substrate should first demonstrate that it can reproduce a known baseline under controlled conditions. The reported results do that. They do not yet show that genomic execution is more efficient or more accurate than dense neural execution. They show that the substrate is computationally serious and experimentally inspectable.

The more informative result is the trace structure. Each inference through the genomic substrate produces a complete mechanistic record—promoter activations, strand bindings, transfer products, accumulator completions, activation firings—that is structurally richer than what post-hoc interpretability methods can recover from dense neural execution. In SDANS, interpretability is a first-class property of the execution model, not an auxiliary analysis.

15.3 Local structural adaptation as a distinct paradigm

The local edit operators demonstrate a qualitatively different approach to model modification. After injecting noise into 33% of synapse weights, the structural retuning procedure recovers full accuracy through a sequence of targeted proposals—each modifying a single synapse, bias, or gate—with formal type-checking before acceptance. This is analogous to how biological systems repair DNA through enzymatic editing rather than whole-genome rewriting. Two properties distinguish this from gradient-based fine-tuning: (i) each edit is type-checked and safety-verified, preventing catastrophic forgetting by construction, and (ii) edit granularity is at the individual connection level, enabling modifications that global gradient flow cannot easily isolate.

15.4 From equivalence to post-neural architecture

With equivalence established, the HelixSim substrate exposes structural properties that dense execution cannot easily replicate:

- **Mechanistic traceability.** Every binding, transfer, and activation is recorded, providing built-in causal attribution.
- **Selective expression.** Gene gating enables input-dependent computation graphs—the genomic analogue of conditional computation.
- **Structural modularity.** The genome is organized into typed, addressable gene complexes that can be composed, silenced, or transplanted.

These properties form the foundation for DOGMA (DNA-Organized Genomic Model Architecture) [10], a post-transformer language model that extends HelixSim with four parallel DNA computation paths, thermodynamic strand attention, chemical reaction networks, and codon translation. The DOGMA-Supreme model (224.7M parameters, 12 blocks) is currently under active training, with validation loss improving from 5.726 to 3.756 over 800 steps (Table 8).

Table 8: DOGMA-Supreme training progress (ongoing).

Step	Train Loss	Valid Loss	Status
0	5.6536	5.7263	Initial
100	3.7525	3.8072	—
200	4.0651	4.0588	—
300	4.0127	3.8033	—
400	3.9188	3.8095	—
500	3.8867	3.8424	—
600	3.8055	3.8487	—
700	4.0784	3.8588	—
800	3.9040	**3.7557**	Best

Notably, DOGMA replaces all traditional weight matrices (`nn.Linear`) with biologically-grounded learnable parameters organized into six component families per block:

Table 9: DOGMA-Supreme parameter composition per block (×12).

Component	Params	Key Parameter Types
DNA Compute (4-path)	8,544,649	$k_{\max}$, α, stoichiometry, WC-matrices
TetraMemory (multi-scale)	8,102,504	Bond projectors, face gates, decay
SDANS Neural Layer	719,040	Algebraic operator weights
Regulation Gate	565,121	Expression/suppression thresholds
Mixer	418,096	Post-compute mixing
Codon Translation	337,408	64→21 codon mapping tables
Per-block total	**18,686,818**	—
Full model	**224,743,786**	12 blocks + embedding + realization

15.5 What this paper does not establish

This manuscript does not establish any of the following:

- superiority over transformers on real language benchmarks,
- asymptotic runtime improvements over optimized dense kernels,
- evidence for biological wet-lab implementation,
- large-scale scaling behavior beyond the reported small benchmarks.

15.6 Why the result still matters

Even with those limitations, the result is important. It shows that a DNA-algebraic substrate can be made operational, measurable, and falsifiable. That is the correct foundation for later work on selective expression, staged regulation, richer double-helix storage, tetrahedral 3D representations, and more ambitious language architectures.

16 Threats to Validity

Following standard methodology for empirical software engineering research [11], we identify potential threats to the validity of our findings and describe the mitigations applied.

16.1 Internal validity

Implementation correctness. A bug in the SDANS compiler or expression engine could artificially produce agreement. We mitigate this through a suite of 126 unit tests (4 skipped for optional dependencies), independent manual verification of the XOR trace against hand computation, and comparison of SDANS outputs against PyTorch's own quantized execution path.

Seed sensitivity. Fixed random seeds (XOR: seed=7, Tiny MLP: seed=42, CharLM: seed=42) ensure reproducibility but could mask initialization sensitivity. The scheduling ablation—in which deterministic, random, and priority firing orders produce bit-identical outputs—confirms that results are invariant to execution ordering for a given trained model.

Precision confound. The $\mathbf{Fix}\langle 8, 8\rangle$ quantization scheme could trivialize equivalence if it destroyed task-relevant information. We verify that quantized neural accuracy matches floating-point accuracy on XOR and Tiny MLP (both 1.000) and preserves relative ordering on CharLM-tiny (0.186), confirming that quantization preserves the computation we aim to simulate.

16.2 External validity

Scale. All benchmarks use small models (27–99 genes, 167–2,130 edges). The simulation theorem holds for arbitrary finite feed-forward networks, but practical concerns at millions of genes remain uncharacterized. The DOGMA-Supreme model (224.7M parameters) represents an initial step toward scale characterization.

Architecture scope. Theorem 6 covers feed-forward networks with piecewise-linear activations. Extension to recurrent architectures, attention mechanisms, and state-space models [8, 9] requires additional compilation rules. The DOGMA architecture addresses this by implementing these components natively in the DNA-algebraic substrate rather than through compilation from neural models.

Task diversity. Three benchmarks spanning boolean logic, classification, and language model-

ing cover different computational regimes but do not exhaust all relevant domains. Generalization to retrieval, reasoning, and multimodal tasks requires further empirical work.

16.3 Construct validity

Agreement metric. We measure exact-match agreement between quantized neural and SDANS outputs—the strongest possible equivalence criterion. Weaker metrics (cosine similarity, top-k overlap) would show equal or higher agreement.

Expression cost. We define EC as active genes plus active edges. Alternative weightings (by codon precision, trace depth, or energy) might yield different efficiency conclusions. We adopt the simplest definition for transparency.

17 Limitations and Future Work

1. **Feed-forward scope.** The simulation theorem covers bounded-precision feed-forward networks. Extending to recurrent, attention-based, and state-space architectures is the immediate theoretical priority.

2. **Runtime overhead.** SDANS is 200×–900× slower than PyTorch dense inference. Three mitigation strategies are under investigation: JIT compilation of the genomic execution graph, selective expression to skip inactive gene complexes, and batched strand operations via tensor parallelism.

3. **Benchmark scale.** Scaling from $\mathcal{O}(10^2)$ genes to $\mathcal{O}(10^6)$ genes remains to be characterized. Memory, compilation time, and trace storage at scale are open engineering challenges.

4. **Quantization ceiling.** While $\mathbf{Fix}\langle 8, 8\rangle$ codons suffice here (max error ≤ 0.002), high-precision applications may need wider codons or mixed-precision scheduling across network depth.

5. **Multimodal extension.** The current substrate operates on scalar sequences. Extension to images (via tetrahedral mesh representation), audio (via waveform codons), and structured 3D data (via linked-tetrahedron volumetric encoding) is a key future direction. We outline a concrete proposal in a companion technical document.

6. **Biological plausibility.** DOGMA's parameters are grounded in biological constants, but training uses gradient-based optimization. Developing purely local learning rules that exploit the genomic substrate's structural properties remains open.

18 Conclusion

HelixSim advances a focused research position: before proposing a genomic replacement for modern AI architectures, one should first show that genomic execution can reproduce bounded neural computation faithfully. SDANS provides such a substrate. It combines typed computational strands, DNA-algebraic operators, fixed-point codons, stochastic regulation, mechanistic traces, and verified local edits into a single execution model. The current implementation and experiments indicate that this model can reproduce quantized neural behavior exactly on the reported small-scale benchmarks while exposing execution structure that dense neural runtimes ordinarily hide.

That result is not the end state. It is the starting point. The next steps are to extend the theorem and implementation to richer model classes, quantify trace-local learning at larger scale, test whether regulated expression yields tangible computational advantages, and explore multimodal extensions including tetrahedral 3D mesh representations. Those questions remain open; this paper establishes the baseline from which they can be studied rigorously.

Reproducibility

The reported implementation includes an SDANS reference module, a benchmark script, and stored experiment artifacts. To reproduce the benchmark suite:

```
pip install -e .
python scripts/helixsim_benchmarks.py
```

All reported runs use fixed seeds in the provided scripts. Results artifact: `artifacts/helixsim_benchmark_results.json`.

References

[1] R. Freund, L. Kari, and G. Păun. DNA computing based on splicing: The existence of universal computers. *Theory of Computing Systems*, 32(1):69–112, 1999.

[2] E. Winfree. *Algorithmic Self-Assembly of DNA*. PhD thesis, California Institute of Technology, 1998.

[3] G. Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems*, 2:303–314, 1989.

[4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. Gomez, L. Kaiser, and I. Polosukhin. Attention Is All You Need. In *Advances in Neural Information Processing Systems*, 2017.

[5] J. SantaLucia, Jr. A unified view of polymer, dumbbell, and oligonucleotide DNA nearest-neighbor thermodynamics. *Proceedings of the National Academy of Sciences*, 95(4):1460–1465, 1998.

[6] D. Y. Zhang and E. Winfree. Control of DNA strand displacement kinetics using toehold exchange. *Journal of the American Chemical Society*, 131(47):17303–17314, 2009.

[7] D. Soloveichik, G. Seelig, and E. Winfree. DNA as a universal substrate for chemical kinetics. *Proceedings of the National Academy of Sciences*, 107(12):5393–5398, 2010.

[8] A. Gu, K. Goel, and C. Ré. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations*, 2022.

[9] A. Gu and T. Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

[10] W. Qin. Evolutor: A Theory of Genomic Computation. Technical report, MapleAI Intelligence Inc., 2026.

[11] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén. *Experimentation in Software Engineering*. Springer, 2012.

[12] I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, and Y. Bengio. Quantized neural networks: Training neural networks with low precision weights and activations. *Journal of Machine Learning Research*, 18(187):1–30, 2017.